

Recorrido del código

`highOrderManufacturedFDAImplicitPETScDistributed`

Generado el 2026-09-29 a partir de
`highOrderManufacturedFDAImplicitPETScDistributed.C` (10145 líneas)

Cómo leer esto. Los listados NO son copias: salen de `\lstinputlisting` apuntando al fuente real, con rangos que este reporte recalcula al regenerarse. El PDF de hoy muestra el código de hoy. Los números de línea del margen son los del archivo.

Índice

1. Qué resuelve este solver	3
2. De qué se compone: tres archivos y una capa	3
3. Andamiaje de PETSc: errores y matriz distribuida	4
4. El offset global de fila	5
5. Nombres de opciones: una sola ortografía, y los de PETSc	6
6. La solución manufacturada	7
7. Bloques de construcción del ensamblado	9
8. Las dos matrices de rigidez	11
9. Resolución del sistema lineal	13
10. Discretización temporal: las dos familias	16
11. El driver de Vm que manejan las EDO	19
12. Reconstrucción a los puntos de integración	20
13. Integración de las EDO, y el paralelismo por hilos	21
14. <code>main()</code> : montaje	23
15. <code>main()</code> : el lazo temporal y los métodos no lineales	25

1. Qué resuelve este solver

Resuelve la ecuación de monodominio con una **solución manufacturada**: se elige de antemano un campo exacto $V_m(x, t)$ y tres estados u_1, u_2, u_3 , se los mete en la ecuación, y lo que sobra se agrega como término fuente. Con eso el error es *medible* y no estimable, que es lo que permite demostrar el orden de precisión — el resultado central del solver.

La ecuación discretizada en el tiempo, con θ eligiendo el esquema ($\theta = 1$ backward Euler, $\theta = 1/2$ Crank–Nicolson), es

$$\left(\frac{M}{\Delta t} - \theta \lambda K\right) V_m^{n+1} = \left(\frac{M}{\Delta t} + (1 - \theta) \lambda K\right) V_m^n + b$$

con M la matriz de masa, K el operador de difusión y b el término fuente más las condiciones de borde. El archivo está organizado casi exactamente en ese orden: primero la infraestructura lineal, después M y K , después la parte no lineal, y al final el lazo temporal.

Dos ejes de paralelismo, y conviene tenerlos separados desde el principio porque el archivo los trata en lugares distintos: **MPI** reparte la malla entre ranks y toca todas las fases, mientras **OpenMP** reparte un lazo de celdas dentro de un rank y sólo toca la integración de las EDO.

2. De qué se compone: tres archivos y una capa

Conviene decirlo antes de mirar un listado, porque los nombres que aparecen en todos ellos no dicen de dónde salen. El solver se construye a partir de **tres** archivos del repositorio y de nada más:

- `highOrderManufacturedFDImplicitPETScDistributed.C`, el recorrido de este documento;
- `createFields.H`, que lee los diccionarios y construye los campos y los operadores;
- `src/highOrderAdapter/highOrderInterp.H`, la capa de alto orden.

Afuera queda OpenFOAM, PETSc, Eigen y **solids4foam**, de donde sale la reconstrucción de alto orden propiamente dicha. Después de sacar del `Make/options` las inclusiones y bibliotecas que ya no se usaban, el binario enlaza tres objetos compartidos: `libsolids4FoamModels`, `libpetsc` y `libOpenFOAM`.

Por qué el código dice “LRE” en todas partes. La reconstrucción la hacía originalmente LRE (Castrillo *et al.*, *Comput Struct* 268:106829, 2022), una biblioteca **serial**: su construcción de stencils retorna antes del intercambio MPI — que está comentado — y los patches de procesador lanzan `NotImplemented`. Es decir que el port a MPI no podía hacerse sobre ella. La reemplaza `movingLeastSquares`, también de `solids4foam`, que sí es paralela de verdad: los stencils llevan **identificadores globales** de celda y cada punto de evaluación hace su propio intercambio de halo, de modo que el llamador no cambia para correr en paralelo. Los nombres `LREInterp_Vm`, `LREInterp_Iion`, `LREInterp_states` y las claves `LRECoeffs*` del diccionario son **históricos**: LRE ya no interviene.

Qué hace la capa, porque casi todo lo que contiene es reenvío de una línea y podría no existir. Cuatro diferencias de contrato la justifican:

1. **Las claves del diccionario.** El solver y todos los casos del árbol de corridas están escritos en la ortografía de LRE (`N`, `Nn`, `weightFunction`, `k`), y la capa las traduce a las de `movingLeastSquares`. Vale la pena saber que `N` es el *grado* del polinomio y `Nn` son las celdas *agregadas* sobre el mínimo, que es el número de términos del desarrollo de Taylor: $(p+1)(p+2)/2$ en 2D y $(p+1)(p+2)(p+3)/6$ en 3D. Con `N = 2` y `Nn = 10` en 2D el stencil tiene $6 + 10 = 16$ celdas.

2. **Los coeficientes de derivadas por debajo del orden que los necesita.** LRE reservaba los arreglos de derivada segunda y tercera siempre, así que el solver liga la referencia y recién guarda el *uso* con `order() >= 2`. `movingLeastSquares` no los construye por debajo del orden requerido y **aborta** si se los piden; la capa devuelve una lista vacía y eso es lo que mantiene correctos los tres bindings incondicionales del solver.
3. **Los pesos de cuadratura cambiaron de significado.** LRE los normalizaba — una cara sumaba 1 — y el llamador multiplicaba por $|S_f|$; los de `solids4foam` son **físicos** y suman $|S_f|$. Para que la diferencia no reescalara un resultado en silencio, los accesorios se llaman `faceQuadWeightPhysical()` y `cellQuadWeightPhysical()`: un sitio portado sin revisar **no compila**.
4. **Una sobrecarga rota.** La forma de `fGrad` que devuelve `autoPtr` llama a un miembro que no existe, así que la capa dimensiona el resultado y usa la de dos argumentos.

Nada de eso desaparece si se borra la capa: se muda a cada consumidor. Y hay dos consumidores, porque el solver del electro usa la misma.

3. Andamiaje de PETSc: errores y matriz distribuida

El solver ensambla en Eigen y resuelve en PETSc, así que lo primero del archivo es el puente entre los dos.

`checkPetscError` convierte un código de retorno de PETSc en un `FatalError` de OpenFOAM con el contexto de la llamada. Sin eso un error de PETSc se propaga como un entero que nadie mira.

`buildDistributedMat` es la pieza central del port a MPI: arma la matriz `MPIAIJ` a partir de los tripletes locales. La matriz es `nLocal × nGlobal` — filas propias, columnas globales — que es el layout que PETSc espera. En serie `gRowStart` vale cero y `nLocal == nGlobal`, así que el camino es el mismo.

```

133 void checkPetscError(const PetscErrorCode ierr, const char* context)
134 {
135     if (ierr)
136     {
137         FatalErrorInFunction
138             << "PETSc call failed in " << context
139             << " with error code " << ierr
140             << exit(FatalError);
141     }
142 }
```

Listing 1: Un código de retorno de PETSc pasa a ser un error legible.

```

155 void buildDistributedMat(const SpMat& A, Mat& out)
156 {
157     const PetscInt nLocalRows = static_cast<PetscInt>(A.rows());
158     const PetscInt nGlobalCols = static_cast<PetscInt>(A.cols());
159     const PetscInt rStart = static_cast<PetscInt>(globalRowStart());
160     const PetscInt rEnd = rStart + nLocalRows;
161
162     std::vector<PetscInt> dNnz(nLocalRows, 0);
163     std::vector<PetscInt> oNnz(nLocalRows, 0);
164     for (PetscInt row = 0; row < nLocalRows; ++row)
165     {
166         for (SpMat::InnerIterator it(A, row); it; ++it)
167         {
168             const PetscInt c = static_cast<PetscInt>(it.col());
169             if (c >= rStart && c < rEnd)
170             {
171                 ++dNnz[row];
172             }
173             else
174             {

```

```

175         ++oNnz[row];
176     }
177 }
178 }
179
180 checkPetscError(MatCreate(PETSC_COMM_WORLD, &out), "MatCreate");
181 checkPetscError
182 (
183     MatSetSizes
184     (
185         out,
186         nLocalRows,
187         nLocalRows,    // local size of the vector this matrix multiplies
188         PETSC_DETERMINE,
189         nGlobalCols
190     ),
191     "MatSetSizes"
192 );
193 checkPetscError(MatSetType(out, MATAIJ), "MatSetType");
194 checkPetscError
195 (
196     MatSeqAIJSetPreallocation
197     (
198         out, 0, dNnz.empty() ? nullptr : dNnz.data()
199     ),
200     "MatSeqAIJSetPreallocation"
201 );
202 checkPetscError
203 (
204     MatMPIAIJSetPreallocation
205     (
206         out,
207         0, dNnz.empty() ? nullptr : dNnz.data(),
208         0, oNnz.empty() ? nullptr : oNnz.data()
209     ),
210     "MatMPIAIJSetPreallocation"
211 );
212 checkPetscError
213 (
214     MatSetOption(out, MAT_NEW_NONZERO_ALLOCATION_ERR, PETSC_FALSE),

```

Listing 2: Tripletas locales a matriz distribuida. Recortado. (*recortado; sigue en el fuente.*)

4. El offset global de fila

`gRowStart` es el desplazamiento de la primera fila de este rank, o sea `globalIndex::localStart()`. Es constante para toda la corrida — una malla, una descomposición — así que se fija una vez en `main()` en vez de arrastrarlo por la firma de cada rutina.

Vale la pena detenerse acá porque es donde vivían cuatro bugs del port, todos del mismo tipo: escribir una columna *local* donde iba una *global*. En serie `gRowStart` es cero y `toGlobal()` es la identidad, así que **ninguno de los cuatro era visible sin correr en paralelo**. El test **M09** compara K entrada por entrada contra el ensamblado serial deshaciendo la permutación con `cellProcAddressing`, y es lo único que puede verlos: un fingerprint de nnz y sumas de fila es invariante a mover una entrada de columna.

Justamente por eso el valor no se lee directo sino a través de `globalRowStart()`, que **se niega a servir el cero inicial**. La asignación ocurre en `main()`, después de construir el interpolador, y todas las inserciones de fila en PETSc la leen. Un camino de código que llegara a una matriz antes de ese punto usaría offset cero *en todos* los ranks, o sea que los ranks > 0 escribirían sus filas encima de las del rank 0: operador corrompido y respuesta plausible pero equivocada, nunca un error. Hoy nada lo hace; el accesor lo vuelve imposible en vez de meramente improbable.

```

96 // Global row offset of this rank's block of cells,
97 // i.e. globalIndex::localStart(). Constant for the whole run (one mesh, one
98 // decomposition), so it is set once in main() rather than threaded through

```

```

99 // every linear-solver signature. Zero in serial.
100 label gRowStart = 0;
101 bool gRowStartSet = false;
102
103 // Read through an accessor that refuses to serve
104 // the initial 0. The assignment happens in main, after the interpolator is
105 // built, and every PETSc row insertion reads it. A future code path that
106 // touched a matrix before that point would offset by 0 on EVERY rank, so
107 // ranks > 0 would write their rows on top of rank 0's: a corrupt operator
108 // and a plausible wrong answer, never an error. Nothing does that today -
109 // this makes it impossible rather than merely unlikely.
110 label globalRowStart()
111 {
112     if (!gRowStartSet)
113     {
114         FatalErrorInFunction
115         << "The global row offset was read before it was set."
116         << " It is assigned once in main from"
117         << " LREInterp_Vm.globalCells().localStart(); reaching a PETSc"
118         << " matrix before that offsets every rank's rows by 0 and"
119         << " corrupts the operator."
120         << exit(FatalError);
121     }
122     return gRowStart;
123 }
124

```

Listing 3: El offset, y el accesor que se niega a servirlo sin asignar.

5. Nombres de opciones: una sola ortografía, y los de PETSc

Cada opción del diccionario tiene exactamente una ortografía, comparada distinguiendo mayúsculas, y cualquier otra cosa detiene la corrida al arrancar con la lista de valores válidos. Lo hace `requireOneOf`, que se llama en `createFields.H` justo después de leer cada clave.

Reemplazó a una capa que pasaba todo a minúsculas y acumulaba alias (`sparselu/lu`, `picard`, `localDiagonal`, `RKT45`, `consistentH0`, `jakobi`...). No era cosmético: así se escondió una respuesta equivocada. Con `linearSolver SparseLU`, el camino de `diagonalIion` forzaba un solve directo, pero el solver cacheado de Picard recibía `preonly` con el PC por defecto, `ilu`: una aplicación de ILU haciéndose pasar por la inversa. Picard convergía al punto fijo del operador equivocado — V_m con un error 4,6 veces mayor, cero pasos no convergidos, ninguna advertencia. `resolvePetscKspPc` resuelve ahora ese par en un solo lugar para los tres caminos PETSc.

Quedan dos vocabularios separados. `linearSolver` es neutral (`GMRES`, `BiCGSTAB`, `SparseLU`); las claves de PETSc usan los nombres propios de PETSc de listas cerradas. `petscKspTypeForLinearSolver` es lo único que traduce de uno a otro, para el valor por defecto de `petscLinearKspType`.

La función de esta sección que importa para interpretar resultados es `petscParallelPcTypeName`.

`ilu` y `lu` no tienen implementación `MPIAIJ`, así que en paralelo PETSc los envuelve en **block-Jacobi**, y el bloque es la partición. Es decir que a `np=1` y a `np=2` no se resuelve con el mismo preconditionador: son dos algoritmos distintos. `hypre` y `gamg` sí son paralelos nativos, pero su *coarsening* también depende de cómo quedó cortada la malla.

Sólo `jacobi` es el mismo operador a cualquier número de ranks, y esa es la razón de que la suite lo fije en todos los tests de invariancia. El test **M07** lo cuantifica: la desviación entre `np=1` y `np=2` es **1.26e-11** con `jacobi` contra **3.60e-05** con `ilu`, un factor de 2.86 millones.

Para producción la respuesta es otra y está en **M18**: ahí lo que importa es el tiempo, y el ranking cambia con el paso de tiempo.

```

359 void requireOneOf
360 (
361     const char* key,

```

```

362     const word& value,
363     std::initializer_list<const char*> valid
364 )
365 {
366     for (const char* v : valid)
367     {
368         if (value == v)
369         {
370             return;
371         }
372     }
373
374     std::string options;
375     for (const char* v : valid)
376     {
377         if (!options.empty())
378         {
379             options += ", ";
380         }
381         options += v;
382     }
383
384     FatalErrorInFunction
385     << "Unknown " << key << ": " << value << nl
386     << "Valid options are " << options.c_str()
387     << " (exact spelling, case-sensitive)."
388     << exit(FatalError);
389 }

```

Listing 4: Una sola ortografía por opción, validada al arrancar.

```

425 void resolvePetscKspPc
426 (
427     const word& linearSolver,
428     const word& petscKspType,
429     const word& petscPcType,
430     word& kspType,
431     word& pcType
432 )
433 {
434     kspType = petscKspType;
435     pcType = petscPcType;
436
437     if (linearSolver == "SparseLU")
438     {
439         kspType = "preonly";
440         pcType = "lu";
441     }
442 }

```

Listing 5: El par (KSP, PC) de un linearSolver, en un solo lugar.

```

476 std::string petscParallelPcTypeName(const word& pcType, std::string& subPc)
477 {
478     const std::string s = std::string(pcType);
479     subPc.clear();
480
481     if (Pstream::parRun() && (s == "ilu" || s == "lu"))
482     {
483         subPc = s;
484         return "bjacobi";
485     }
486
487     return s;
488 }

```

Listing 6: La sustitución que hace que ilu deje de ser ilu en paralelo.

6. La solución manufacturada

Acá está lo que hace que este solver pueda demostrar un orden en vez de estimarlo. `exactVm` y las tres `exactU` dan el campo elegido; `computeF` y `computeG` son sus factores espaciales, escritos por

dimensión.

`vmSourcePDE` es el término que se agrega a la ecuación para que ese campo sea solución exacta: se obtiene metiendo la solución elegida en el operador y despejando lo que falta.

`computeBeta` lee `conductivity[0]`, o sea la celda cero, y eso sólo es válido con conductividad **uniforme**: en una malla descompuesta la celda cero es una celda física distinta en cada rank. No es un item abierto sino un **supuesto asumido** — este solver existe para este caso manufacturado, donde D es uniforme por diseño. Queda registrado acá porque es lo primero que habría que tocar si alguna vez entraran fibras, que hacen D no uniforme por construcción.

Hay dos soluciones manufacturadas, y la clave `manufacturedSolution` elige. La de siempre, **sines**, es $\sqrt{1+t} \cos(\pi x) \cos(2\pi y) \dots$: suave y no polinómica, así que ninguna reconstrucción de acá es exacta sobre ella y el orden *espacial* medido es el del esquema.

La otra, **uniform**, tiene todos los factores espaciales iguales a uno. Existe por una razón puntual: toda reconstrucción, cuadratura y flujo de este solver es exacto sobre un campo constante, así que el error espacial es cero a nivel de redondeo y el error contra la solución exacta es **puramente temporal**. Es la única forma de ajustar un orden temporal contra la solución analítica en vez de contra otra solución numérica, y es la vía B de **M02**. A malla fija con **sines** eso es imposible: el error espacial ($2,4 \times 10^{-5}$ en hexa $N = 40$) entierra al temporal, que para BDF4 es $\sim 10^{-10}$.

Lo que cuesta, dicho para que nadie lea de más el resultado: el laplaciano de una constante es cero, así que bajo **uniform** la difusión no participa y lo que se verifica es la integración temporal del sistema de reacción. La difusión la cubre **M02** por autoconvergencia. Y `computeBeta` devuelve cero en ese modo, porque no hay laplaciano que cancelar.

```
1472 scalar exactVm(const point& p, const scalar t, const label dim)
1473 {
1474     return std::sqrt(1.0 + t)*computeF(p, dim);
1475 }
```

Listing 7: El campo exacto.

```
1419 scalar computeF(const point& p, const label dim)
1420 {
1421     // Exactly representable by every reconstruction here, so the spatial
1422     // error vanishes and what is left is the temporal error.
1423     if (mmsProfile == mmsUniform)
1424     {
1425         return 1.0;
1426     }
1427
1428     const scalar pi = constant::mathematical::pi;
1429
1430     if (dim == 1)
1431     {
1432         return std::cos(pi*p.x());
1433     }
1434     else if (dim == 2)
1435     {
1436         return std::cos(pi*p.x())*std::cos(2.0*pi*p.y());
1437     }
1438
1439     return
1440         std::cos(pi*p.x())
1441         *std::cos(2.0*pi*p.y())
1442         *std::cos(3.0*pi*p.z());
1443 }
```

Listing 8: El factor espacial, y dónde el modo uniform lo vuelve 1.

```
1566 scalar vmSourcePDE
1567 (
1568     const scalar V,
1569     const scalar u1,
1570     const scalar u2,
1571     const scalar u3,
1572     const scalar beta,
```

```

1573     const scalar chiVal,
1574     const scalar CmVal
1575 )
1576 {
1577     return -ionicCurrentPDE(V, u1, u2, u3, beta, chiVal, CmVal);
1578 }

```

Listing 9: El término que lo convierte en solución.

```

1584 scalar computeBeta(const volTensorField& conductivity, const label dim)
1585 {
1586     // A constant has zero Laplacian, so there is nothing for beta to
1587     // cancel and the ionic current reduces to the term that supplies
1588     // dV/dt.
1589     if (mmsProfile == mmsUniform)
1590     {
1591         return 0.0;
1592     }
1593
1594     const tensor& D = conductivity[0];
1595     const scalar pi2 = sqr(constant::mathematical::pi);
1596
1597     if (dim == 1)
1598     {
1599         return -pi2*D.xx();
1600     }
1601     else if (dim == 2)
1602     {
1603         return -pi2*(D.xx() + 4.0*D.yy());
1604     }
1605
1606     return -pi2*(D.xx() + 4.0*D.yy() + 9.0*D.zz());
1607 }

```

Listing 10: Lee la celda 0: sólo vale con D uniforme.

7. Bloques de construcción del ensamblado

Antes de las dos matrices de rigidez vienen las piezas que las dos comparten.

`addTripletIfNeeded` es el punto por el que pasa *toda* entrada de la matriz. Descarta las que valen menos que `SMALL`, lo que hace que el patrón de esparsidad dependa del valor y, por lo tanto, de la partición al nivel del redondeo.

`addCellGradientDotCoeffs` agrega la fila de gradiente de una celda proyectada sobre un vector. Usa `globalCellStencils()`, y ahí vivió durante meses el que se creía el problema abierto más grande del proyecto: sobre las celdas pegadas a un corte de procesador el stencil **perdía un miembro** (17 en serie contra 16 en paralelo). Como el ajuste MLS se rehace sobre el conjunto reducido, no cambiaba sólo el término que faltaba — **refitaba toda la fila**. Eso era lo que hacía fallar a **M11** y, río abajo, a **M15**, y se había dado por un ítem de `solids4foam` que se mide y no se arregla.

No lo era. La causa estaba del lado del caso: la clave `Nn` del diccionario son las celdas que se AGREGAN sobre el mínimo que exige el grado, y los barridos la venían escribiendo como si fuera el TOTAL. La biblioteca sumaba el mínimo otra vez, así que los estencils medían `2 min + 10` en vez de `min + 10` — 16/22/30 celdas en 2D en vez de 13/16/20, un 50 % más grandes de lo pretendido — y el halo no alcanzaba a cubrirlos. Con el tamaño correcto el stencil es invariante a la partición: **M11 mide 0 stencils distintos y M15 baja de 4.981e-06 a 6.953e-15**, los dos en PASS. La suite quedó 24/24.

`exchangeCoupledFaceRows` es la que permite que el término de estabilización cruce un corte: intercambia con el rank vecino las filas que hacen falta del otro lado. Es **colectiva**, y está deliberadamente fuera del lazo de chunks — adentro se trabaría en cuanto dos ranks tuvieran una cantidad distinta de chunks.

```

2308 void addTripletIfNeeded
2309 (
2310     std::vector<Triplet>& triplets,
2311     const label row,
2312     const label col,
2313     const scalar value
2314 )
2315 {
2316     if (mag(value) > SMALL)
2317     {
2318         triplets.emplace_back(row, col, value);
2319     }
2320 }

```

Listing 11: Por acá pasa toda entrada de la matriz.

```

2703 void addCellGradientDotCoeffs
2704 (
2705     std::vector<Triplet>& triplets,
2706     const label row,
2707     const scalar scale,
2708     const label cellI,
2709     const vector& d,
2710     const highOrderInterp& LREInterp
2711 )
2712 {
2713     if (mag(scale) <= SMALL)
2714     {
2715         return;
2716     }
2717
2718     // Matrix columns are global cell indices.
2719     const globalIndex& gc = LREInterp.globalCells();
2720
2721     const CompactListList<label>& stencils = LREInterp.globalCellStencils();
2722     const CompactListList<vector>& gradCoeffs = LREInterp.QRGradCoeffs();
2723     // CompactListList::operator[] returns a
2724     // UList VIEW by value, not a List. Binding it to a const labelList&
2725     // compiles but yields an EMPTY list, silently dropping the whole
2726     // stencil. Bind to the returned view type instead.
2727     const UList<label> curStencil = stencils[cellI];
2728     const label selfCoeffI = curStencil.size();
2729
2730     forAll(curStencil, cI)
2731     {
2732         addTripletIfNeeded
2733         (
2734             triplets,
2735             row,
2736             curStencil[cI],
2737             scale*(gradCoeffs[cellI][cI] & d)
2738         );
2739     }
2740
2741     addTripletIfNeeded
2742     (
2743         triplets,
2744         row,
2745         gc.toGlobal(cellI),
2746         scale*(gradCoeffs[cellI][selfCoeffI] & d)
2747     );
2748 }

```

Listing 12: La fila de gradiente. Recortada.

```

2904 void exchangeCoupledFaceRows
2905 (
2906     const fvMesh& mesh,
2907     const std::function
2908     <
2909         void(const label patchI, const label faceI, labelList&, scalarField&)
2910     >& buildRow,
2911     List<labelList>& nbrCols,
2912     List<scalarField>& nbrCoeffs
2913 )

```

```

2914 {
2915     nbrCols.setSize(mesh.nBoundaryFaces());
2916     nbrCoeffs.setSize(mesh.nBoundaryFaces());
2917
2918     if (!Pstream::parRun())
2919     {
2920         return;
2921     }
2922
2923     PstreamBuffers pBufs(Pstream::commsTypes::nonBlocking);
2924
2925     forAll(mesh.boundary(), patchI)
2926     {
2927         const fvPatch& patch = mesh.boundary()[patchI];
2928         const processorFvPatch* ppPtr = isA<processorFvPatch>(patch);
2929
2930         if (!ppPtr)
2931         {
2932             continue;
2933         }
2934
2935         List<labelList> sendCols(patch.size());
2936         List<scalarField> sendCoeffs(patch.size());
2937
2938         forAll(patch, faceI)
2939         {
2940             buildRow(patchI, faceI, sendCols[faceI], sendCoeffs[faceI]);
2941         }
2942
2943         UOPstream toNbr(ppPtr->neighbProcNo(), pBufs);
2944         toNbr << sendCols << sendCoeffs;
2945     }
2946
2947     pBufs.finishedSends();
2948
2949     forAll(mesh.boundary(), patchI)
2950     {
2951         const fvPatch& patch = mesh.boundary()[patchI];
2952         const processorFvPatch* ppPtr = isA<processorFvPatch>(patch);
2953
2954         if (!ppPtr)
2955         {
2956             continue;
2957         }
2958
2959         List<labelList> recvCols;
2960         List<scalarField> recvCoeffs;
2961
2962         UIPstream fromNbr(ppPtr->neighbProcNo(), pBufs);
2963         fromNbr >> recvCols >> recvCoeffs;

```

Listing 13: Intercambio de filas a través de un corte. Recortada. (*recortado; sigue en el fuente.*)

8. Las dos matrices de rigidez

El solver tiene dos ensamblados del operador de difusión y la tríada del caso elige cuál corre.

`assembleStandardOrthogonalStiffnessMatrix` es el de bajo orden: flujo de dos puntos entre celdas vecinas. Usa el promedio $\frac{1}{2}(D_{own} + D_{nei})$ como tensor de cara.

`assembleHighOrderStiffnessMatrix` es el de alto orden, sobre el stencil MLS con cuadratura en las caras. Acá estuvo el defecto más grande que encontró la verificación: usaba `conductivity[own]` en las dos filas de una cara interna, en vez del promedio. Con D uniforme los dos tensores coinciden y se cancela idénticamente, así que fue invisible durante toda la vida del código; con D no uniforme hacía que K cambiara un 21 % al descomponer la malla, porque al cortar, cada rank es dueño de su lado de la cara. Está corregido, y M15 lo mide: de 2.108e-01 a 4.736e-15.

Las dos ensamblan por chunks de caras, y la clave del diccionario `assemblyFaceChunk` controla el

tamaño. Es control de capacidad solamente: el resultado no debe depender de él, y **M12** lo verifica — con tolerancia y no bit a bit, porque cambiar el tamaño de bloque cambia el orden de suma en punto flotante.

```

3175 void assembleStandardOrthogonalStiffnessMatrix
3176 (
3177     const fvMesh& mesh,
3178     const volTensorField& conductivity,
3179     const highOrderInterp& LREInterp,
3180     const scalar stabilisationAlpha,
3181     SpMat& K
3182 )
3183 {
3184     const vectorField& C = mesh.C();
3185     const scalarField& V = mesh.V();
3186     const labelUList& owner = mesh.owner();
3187     const labelUList& neighbour = mesh.neighbour();
3188
3189     // Local rows, GLOBAL columns, as in
3190     // assembleHighOrderStiffnessMatrix. This assembly still used local
3191     // columns for its orthogonal terms - and, through
3192     // addCellGradientDotCoeffs, global ones for its stabilisation terms -
3193     // which agree only in serial.
3194     const globalIndex& gc = LREInterp.globalCells();
3195
3196     std::vector<Triplet> triplets;
3197     triplets.reserve(4*mesh.nFaces());
3198
3199     forAll(neighbour, faceI)
3200     {
3201         const label own = owner[faceI];
3202         const label nei = neighbour[faceI];
3203
3204         const tensor Df = 0.5*(conductivity[own] + conductivity[nei]);
3205         const vector dPN = C[nei] - C[own];
3206         const scalar a = orthogonalDiffusionCoeff(mesh.Sf()[faceI], dPN, Df);
3207
3208         addTripletIfNeeded(triplets, own, gc.toGlobal(own), -a/max(V[own], SMALL));
3209         addTripletIfNeeded(triplets, own, gc.toGlobal(nei), a/max(V[own], SMALL));
3210         addTripletIfNeeded(triplets, nei, gc.toGlobal(own), a/max(V[nei], SMALL));
3211         addTripletIfNeeded(triplets, nei, gc.toGlobal(nei), -a/max(V[nei], SMALL));
3212
3213         // The tangential flux a two-point form would drop. grad V|_f is taken
3214         // as the average of the two cell gradients, which is what OpenFOAM's
3215         // correction does; here each of them is expanded into the matrix
3216         // through its reconstruction row, so the term is IMPLICIT.
3217         const vector w =
3218             nonOrthFluxVector(mesh.Sf()[faceI], dPN, Df);
3219         const scalar areaHalf = 0.5*(mag(mesh.Sf()[faceI]) + VSMALL);
3220
3221         for (const label cellI : {own, nei})
3222         {
3223             addCellGradientDotCoeffs
3224             (
3225                 triplets, own, areaHalf/max(V[own], SMALL), cellI, w, LREInterp
3226             );
3227             addCellGradientDotCoeffs
3228             (
3229                 triplets, nei, -areaHalf/max(V[nei], SMALL), cellI, w, LREInterp
3230             );
3231         }
3232     }
3233
3234     const surfaceVectorField& Cf = mesh.Cf();

```

Listing 14: Bajo orden: promedio en la cara. Recortado. (recortado; sigue en el fuente.)

```

3594 void assembleHighOrderStiffnessMatrix
3595 (
3596     const fvMesh& mesh,
3597     const volTensorField& conductivity,
3598     const highOrderInterp& LREInterp,
3599     const scalar stabilisationAlpha,
3600     const label tripletsPerFaceReserve,
3601     const label faceChunk,

```

```

3602     SpMat& K
3603 )
3604 {
3605     // Chunked assembly: avoids accumulating ~434M triplets in a single
3606     // std::vector (which would peak at ~19 GB transient due to repeated
3607     // capacity-doubling reallocations on large 3D unstructured tet
3608     // meshes). The chunked variant bounds the triplet buffer to a fixed
3609     // size per chunk and folds each chunk into the global K via sparse
3610     // matrix addition. Numerically equivalent to the monolithic version:
3611     // K = sum_i K_chunk_i, and addTripletIfNeeded filters by individual
3612     // value magnitude (not by accumulated value), so the chunking is
3613     // transparent to the final result.
3614     const bool twoD = mesh.nGeometricD() == 2;
3615     const vectorField& C = mesh.C();
3616     const surfaceVectorField& Cf = mesh.Cf();
3617     const CompactListList<point>& faceQP = LREInterp.faceQuadPoints();
3618     const CompactListList<scalar>& faceQW = LREInterp.faceQuadWeightPhysical();
3619     const CompactListList<label>& faceStencils = LREInterp.globalFaceStencils();
3620     const List<CompactListList<vector>>& faceGradCoeffs =
3621         LREInterp.QRGradFaceGPCoeffs();
3622
3623     const scalarField& V = mesh.V();
3624     const labelUList& owner = mesh.owner();
3625     const labelUList& neighbour = mesh.neighbour();
3626
3627     const label nCells = mesh.nCells();
3628     // Local rows, global columns (MPIAIJ layout).
3629     const globalIndex& gc = LREInterp.globalCells();
3630     const label nGlobalCells = gc.totalSize();
3631     const label nInternalFaces = neighbour.size();
3632
3633     std::vector<Triplet> triplets;
3634     // Reserve for a full chunk without forcing the tet/stabilised worst
3635     // case on flux-only hex runs. The cap is selected by the caller from
3636     // memoryOptimization controls; it changes capacity only, not assembly.
3637     triplets.reserve(faceChunk * tripletsPerFaceReserve);
3638
3639     SpMat Klocal(nCells, nGlobalCells);
3640     bool kInitialised = false;
3641
3642     auto flushChunk =
3643     [&]()
3644     {
3645         if (triplets.empty())
3646         {
3647             return;
3648         }
3649         Klocal.setZero();
3650         Klocal.setFromTriplets(triplets.begin(), triplets.end());
3651         Klocal.makeCompressed();
3652
3653         if (!kInitialised)

```

Listing 15: Alto orden, sobre el stencil MLS. Recortado. (recortado; sigue en el fuente.)

9. Resolución del sistema lineal

`solveSparseSystem` despacha entre PETSc y Eigen según `linearSolverBackend`. En paralelo sólo el camino de PETSc sirve, y ahora eso **se hace cumplir** en vez de sólo estar comentado: la matriz es $n_{\text{Local}} \times n_{\text{Global}}$, o sea rectangular, y las factorizaciones de Eigen necesitan una matriz cuadrada. Sin el guard, `SparseLU` moría en un `eigen_assert` que nombra un archivo interno de la librería — ni este solver, ni la clave del diccionario que lo eligió, ni la palabra “paralelo”. Medido a $np=2$: el error ahora dice 200 local rows and 400 global columns.

Hay además un GMRES escrito a mano (`solveGMRES`) que queda como camino de respaldo cuando no se usa PETSc. Ése tiene su propio guard, y por un motivo **distinto**: no hay matriz que pueda ser rectangular, pero sus productos internos ($V[i].\text{dot}(w)$, $w.\text{norm}()$) los calcula Eigen sobre el vector *local* y nunca se reducen. Cada rank armaría su propia base de Krylov y su propio test de

convergencia, mientras el producto matriz-vector que llama *sí* es colectivo (`MatMult` de PETSc). Dos ranks que salgan del lazo en distinta iteración se traban ahí: no es un abort, es un cuelgue.

Un tercer punto de la misma familia: `usesPetscBackend` **valida** el nombre en vez de tratarlo como una lista blanca de dos entradas cuyo `else` era Eigen. Cualquier valor que no fuera `petsc/ksp` — un typo en un diccionario, por ejemplo — mudaba la corrida entera al backend serial sin una línea en el log. Es la misma falla que `nonlinearMethod Picrad` en el solver electro, que colapsaba una matriz de métodos sobre un solo método.

Qué solver de Krylov corresponde lo decide la simetría del operador, y está medida en **M10**: el de bajo orden es simétrico a $8.4e-16$, el de alto orden **no** — 7.9 % de asimetría relativa, con 300 de 4292 entradas sin transpuesta. Con la triada de alto orden hay que usar `gmres`. `cg` no falla siempre ahí, que es lo que lo hace peligroso: sobrevive mientras le alcancen pocas iteraciones y se cae cuando necesita más (**E13** lo muestra fallando con `jacobi` y `gamg`).

```
401 bool usesPetscBackend(const word& backend)
402 {
403     requireOneOf("linear solver backend", backend, {"PETSc", "Eigen"});
404     return backend == "PETSc";
405 }
```

Listing 16: El nombre del backend, validado.

```
4351 EigVec solveSparseSystem
4352 (
4353     const SpMat& A,
4354     const EigVec& b,
4355     const word& linearSolverBackend,
4356     const word& linearSolver,
4357     const word& petscKspType,
4358     const word& petscPcType,
4359     const label petscRestart,
4360     const word& petscOptionsPrefix,
4361     const bool petscUseOptions,
4362     const scalar tol,
4363     const label maxIter,
4364     label& linearIterations,
4365     scalar& linearError
4366 )
4367 {
4368     if (usesPetscBackend(linearSolverBackend))
4369     {
4370         word kspType;
4371         word pcType;
4372         resolvePetscKspPc
4373         (
4374             linearSolver, petscKspType, petscPcType, kspType, pcType
4375         );
4376
4377         PetscKspMatrixSolver solver;
4378         solver.reset
4379         (
4380             A,
4381             kspType,
4382             pcType,
4383             tol,
4384             maxIter,
4385             petscRestart,
4386             petscOptionsPrefix,
4387             petscUseOptions
4388         );
4389
4390         return solver.solve(b, linearIterations, linearError);
4391     }
4392
4393     // The Eigen backend is serial only, and the
4394     // comment above this function said so without enforcing it. K carries
4395     // LOCAL rows and GLOBAL columns (the MPIAIJ layout, see
4396     // assembleStiffnessMatrix), so in parallel it is rectangular by design
4397     // and Eigen's SparseLU/BiCGSTAB die on an eigen_assert naming a file
4398     // inside the library - not this solver, not the key that selected them,
4399     // and not the word "parallel".
```

```

4400     if (Pstream::parRun())
4401     {
4402         FatalErrorInFunction
4403             << "linearSolverBackend = " << linearSolverBackend
4404             << " selects the Eigen backend, which is serial only." << nl
4405             << "The assembled matrix has " << A.rows()
4406             << " local rows and " << A.cols()
4407             << " global columns, so Eigen would be handed a rectangular"
4408             << " system." << nl
4409             << "Set 'linearSolverBackend PETSc;' to run on "
4410             << Pstream::nProcs() << " ranks."

```

Listing 17: El despacho entre backends, con el guard de paralelo. Recortado. *(recortado; sigue en el fuente.)*

```

4272 EigVec solveSparseSystemEigen
4273 (
4274     const SpMat& A,
4275     const EigVec& b,
4276     const word& linearSolver,
4277     const scalar tol,
4278     const label maxIter,
4279     label& linearIterations,
4280     scalar& linearError
4281 )
4282 {
4283     if (linearSolver == "SparseLU")
4284     {
4285         Eigen::SparseLU<SpMat> solver;
4286         solver.analyzePattern(A);
4287         solver.factorize(A);
4288
4289         if (solver.info() != Eigen::Success)
4290         {
4291             FatalErrorInFunction
4292                 << "SparseLU factorization failed"
4293                 << exit(FatalError);
4294         }
4295
4296         EigVec x = solver.solve(b);
4297         linearIterations = 1;
4298         linearError = 0.0;
4299
4300         if (solver.info() != Eigen::Success)
4301         {
4302             FatalErrorInFunction
4303                 << "SparseLU solve failed"
4304                 << exit(FatalError);
4305         }
4306
4307         return x;
4308     }
4309     else if (linearSolver == "BiCGSTAB")
4310     {
4311         Eigen::BiCGSTAB<SpMat, Eigen::IncompleteLUT<scalar>> solver;
4312         solver.setTolerance(tol);
4313         solver.setMaxIterations(maxIter);
4314         solver.compute(A);
4315
4316         if (solver.info() != Eigen::Success)
4317         {
4318             FatalErrorInFunction
4319                 << "BiCGSTAB setup failed"
4320                 << exit(FatalError);
4321         }
4322
4323         EigVec x = solver.solve(b);
4324         linearIterations = solver.iterations();
4325         linearError = solver.error();
4326
4327         if (solver.info() != Eigen::Success)
4328         {
4329             FatalErrorInFunction
4330                 << "BiCGSTAB solve failed"

```

Listing 18: El camino serial. Recortado. (*recortado; sigue en el fuente.*)

10. Discretización temporal: las dos familias

El solver ofrece dos familias de integrador temporal, y `implicitScheme` elige entre ellas. Con el orden que cada una debe entregar:

<code>implicitScheme</code>	orden	estabilidad
<code>backwardEuler</code>	$O(\Delta t)$	A y L-estable
<code>crankNicolson</code>	$O(\Delta t^2)$	A-estable, no L-estable
<code>BDF1</code>	idéntico a <code>backwardEuler</code> , bit a bit	A y L
<code>BDF2</code>	$O(\Delta t^2)$	A y L-estable
<code>BDF3</code>	$O(\Delta t^3)$	$A(86,03^\circ)$
<code>BDF4</code>	$O(\Delta t^4)$	$A(73,35^\circ)$

`timeSchemeCoeffs` traduce el nombre a los coeficientes. Lo que hay que ver en esa función es que **las dos familias terminan en el mismo par de operadores**: $A = (a_0/\Delta t)M - \theta L$ y $B = M/\Delta t + (1 - \theta)L$. BDF cambia *sólo* el escalar que multiplica a M , así que el ensamblado, el patrón de esparsidad y la factorización cacheada quedan intactos. Y el historial entra por un *único* producto por la matriz de masa, porque la combinación $H^n = -\sum_{j \geq 1} a_j V^{n+1-j}$ se arma como campo antes de aplicar B : un paso BDF- k cuesta un producto matriz-vector, no k .

BDF5 y BDF6 no se ofrecen. Pasada la segunda barrera de Dahlquist ningún multipaso es A-estable, y la cuña se cierra de 86° y 73° en BDF3/4 a $51,8^\circ$ y $17,8^\circ$ en BDF5/6.

El arranque es la parte que engaña. Un método de k pasos necesita valores iniciales precisos a $O(\Delta t^k)$ respecto del sistema *semi-discreto*, no de la PDE. Sembrar con la solución manufacturada da $O(\tau_h \Delta t)$ — primer orden — porque la solución exacta de la PDE no satisface las ecuaciones semi-discretas. Nada avisa: la corrida termina bien y sólo el orden sale mal, que es por lo que `bdfStartup` tiene hoy **ESDIRK3 por defecto**.

Con ese arranque los órdenes son los nominales. Medido a $N = 40$ por las dos vías que usa la suite — **M02**, autoconvergencia de Cauchy sobre la solución de senos con la difusión activa, y su vía B, error contra la solución analítica sobre la MMS uniforme:

<code>implicitScheme</code>	nominal	Cauchy, vía A	analítica, vía B
<code>backwardEuler</code> y <code>BDF1</code>	1	1,000	1,000
<code>crankNicolson</code>	2	1,994	2,000
<code>BDF2</code>	2	1,979	1,992
<code>BDF3</code>	3	2,932	2,971
<code>BDF4</code>	4	3,890	3,941

`exact` y `constant` se conservan, pero **sólo** para que el costo de un mal arranque siga siendo medible: es lo que hace **M20**, aislando el efecto como $e(\Delta t) = \|V_{\text{arranque}} - V_{\text{ESDIRK3}}\|$ al mismo Δt y la misma malla, donde el error propio de BDF3 se cancela. Los dos modos resultan $O(\Delta t)$, y difieren donde la teoría dice: el de `constant` no cambia al refinar la malla (razón 1,00) y el de `exact` cae 15,5 veces por refinamiento, porque su coeficiente es τ_h .

Por eso existe `esdirk3Tableau`, y **sólo** por eso: ESDIRK no es seleccionable como esquema. Es autoarrancable, que es lo que el arranque pide, pero tiene orden de etapa 2 y sufre reducción de

orden — ESDIRK4 midió 3,03 donde BDF4 llega a 3,89; esa cifra de ESDIRK4 es anterior al arreglo del lazo temporal y no se volvió a medir — y cada etapa reintegra las EDO, que es el 74–99 % del tiempo en el solver fisiológico.

```

1821 TimeSchemeCoeffs timeSchemeCoeffs(const word& scheme)
1822 {
1823     TimeSchemeCoeffs c;
1824
1825     if (scheme == "backwardEuler")
1826     {
1827         c.theta = 1.0;
1828         c.bdfOrder = 0;
1829         return c;
1830     }
1831
1832     if (scheme == "crankNicolson")
1833     {
1834         c.theta = 0.5;
1835         c.bdfOrder = 0;
1836         return c;
1837     }
1838
1839     // BDF is fully implicit in K, so theta = 1, and the source is wanted at
1840     // t^{n+1} only - one source evaluation per step FEWER than
1841     // Crank-Nicolson needs.
1842     c.theta = 1.0;
1843
1844     if (scheme == "BDF1")
1845     {
1846         c.bdfOrder = 1;
1847         c.a[0] = 1.0;
1848         c.a[1] = -1.0;
1849     }
1850     else if (scheme == "BDF2")
1851     {
1852         c.bdfOrder = 2;
1853         c.a[0] = 3.0/2.0;
1854         c.a[1] = -2.0;
1855         c.a[2] = 1.0/2.0;
1856     }
1857     else if (scheme == "BDF3")
1858     {
1859         c.bdfOrder = 3;
1860         c.a[0] = 11.0/6.0;
1861         c.a[1] = -3.0;
1862         c.a[2] = 3.0/2.0;
1863         c.a[3] = -1.0/3.0;
1864     }
1865     else if (scheme == "BDF4")
1866     {
1867         c.bdfOrder = 4;
1868         c.a[0] = 25.0/12.0;
1869         c.a[1] = -4.0;
1870         c.a[2] = 3.0;
1871         c.a[3] = -4.0/3.0;
1872         c.a[4] = 1.0/4.0;
1873     }
1874     else
1875     {
1876         FatalErrorInFunction
1877             << "Unknown implicitScheme: " << scheme << nl
1878             << "Valid options are backwardEuler, crankNicolson, BDF1,"
1879             << " BDF2, BDF3 and BDF4." << nl
1880             << "ESDIRK is not a scheme here: it exists only to supply the"

```

Listing 19: Los coeficientes de las dos familias. Recortado. (*recortado; sigue en el fuente.*)

```

1781 EsdirkTableau esdirk3Tableau()
1782 {
1783     EsdirkTableau tb;
1784     tb.nStages = 4;
1785     tb.order = 3;
1786     tb.gamma = 0.435866521508458999416;
1787
1788     forAll(tb.c, i)

```

```

1789 {
1790     tb.c[i] = 0.0;
1791     forAll(tb.a[i], j)
1792     {
1793         tb.a[i][j] = 0.0;
1794     }
1795 }
1796
1797 const scalar g = tb.gamma;
1798
1799 tb.c[0] = 0.0;
1800 tb.c[1] = 2.0*g;
1801 tb.c[2] = 0.6;
1802 tb.c[3] = 1.0;
1803
1804 tb.a[1][0] = g;
1805 tb.a[1][1] = g;
1806
1807 tb.a[2][0] = 0.257648246066427245800;
1808 tb.a[2][1] = -0.093514767574886245216;
1809 tb.a[2][2] = g;
1810
1811 tb.a[3][0] = 0.187641024346723825161;
1812 tb.a[3][1] = -0.595297473576954948048;
1813 tb.a[3][2] = 0.971789927721772123471;
1814 tb.a[3][3] = g;
1815
1816 return tb;
1817 }

```

Listing 20: La tabla del arranque, derivada de las condiciones de orden.

```

8487 // The ESDIRK startup step. This runs ONLY for
8488 // the first bdfOrder-1 steps of a BDF run, to build the history; it is
8489 // not reachable as a scheme in its own right (see the note on
8490 // needEsdirkOperator above for why not).
8491 //
8492 // Stage 1 is explicit, Y_1 = V^n. For i >= 2 the stage solves
8493 //
8494 // A_g Y_i = (1/g) [ B V^n + sum_{j<i} a_ij F_j ] + bc(t_i) + s(Y_i)
8495 //
8496 // with A_g = M/(g dt) - lapScale K, B = M/dt, t_i = t + c_i dt, and
8497 //
8498 // F_j = lapScale K Y_j + bc(t_j) + s(Y_j).
8499 //
8500 // Note bc enters with coefficient 1 while the B and F terms carry 1/g:
8501 // that falls out of dividing the stage equation by g dt. Getting it
8502 // wrong would be invisible on this MMS suite, where every case is
8503 // zeroGradient and bc is identically zero - hence this note.
8504 //
8505 // The shape is the same one-step equation the Picard branch below
8506 // solves, which is why one matrix, one cached factorisation and one
8507 // fixed-point iteration serve every stage.
8508 //
8509 // Stiffly accurate (c_s = 1, b = last row): V^{n+1} = Y_s, the last
8510 // stage IS the answer and there is no separate combination step.
8511 //
8512 // Only Picard is wired up. JFNK and diagonalIion would each need their
8513 // per-stage Jacobian machinery threaded through, which is separate
8514 // work; an unknown scheme name is a FatalError upstream, so nothing
8515 // reaches here by accident.
8516 // ESDIRK runs only while it is supplying the first bdfOrder-1 starting
8517 // values. After that the history is full and BDF takes over for the
8518 // rest of the run - which, for anything but the opening steps, is the
8519 // whole run.
8520 const bool esdirkThisStep =
8521     (bdfOrder > 1 && esdirkStartup && nSteps < bdfOrder - 1);

```

Listing 21: Dónde el arranque ESDIRK es alcanzable, y sólo ahí.

11. El driver de Vm que manejan las EDO

Las EDO de estado se integran con sub-pasos *dentro* del paso de la PDE, y en cada sub-paso necesitan un V_m en un instante intermedio que el esquema no conoce. Cómo se rellena ese hueco **acota el orden de todo el solver**: una recta entre los dos extremos es $O(\Delta t^2)$, y ningún integrador temporal por encima de segundo orden puede mostrar el suyo detrás de eso.

No es una salvedad teórica, está medido. La clave `stateODEDriver` conmuta entre las dos opciones — `history`, el interpolante por los niveles que el esquema ya guarda, y `linear`, la rampa vieja — y `M21` corre BDF4 con cada una sin cambiar nada más: la rampa ajusta **2,04** contra **3,89** del interpolante, con el error del par más fino mayor por un factor 44,7. El 2,04 es exactamente lo que predice la cuenta: la rampa le erra por $O(\Delta t^2)$ punto a punto, que integrado sobre el paso es $O(\Delta t^3)$ local y $O(\Delta t^2)$ global.

`history` es el valor por defecto; `linear` existe para que ese costo siga siendo medible, porque un orden capado es todo lo que la corrida reporta — no hay error ni advertencia. El camino `gaussPointODE` usa siempre la rampa: subirlo obligaría a guardar cada nivel *en cada punto de cuadratura*.

`interpolateVmDriver` evalúa el polinomio de Lagrange a través de los voltajes que el esquema exterior ya calculó: dos nodos para las familias θ y BDF1, $k + 1$ niveles de historial para BDF- k , y los valores de etapa para el arranque ESDIRK. No cuesta ninguna evaluación extra — los nodos ya existen. El caso de dos nodos está escrito aparte y no pasa por el bucle, que es lo que garantiza que `backwardEuler` y `crankNicolson` sigan dando resultados idénticos bit a bit.

```
5482 inline scalar interpolateVmDriver
5483 (
5484     const VmDriverNodes& nd,
5485     const scalar s
5486 )
5487 {
5488     // Written out for two nodes rather than left to the general loop: this
5489     // is the path every non-ESDIRK scheme takes, and spelling it out is
5490     // what guarantees bit-identical results, not just equal ones.
5491     if (nd.nNodes <= 2)
5492     {
5493         return (1.0 - s)*nd.v[0] + s*nd.v[1];
5494     }
5495
5496     scalar result = 0.0;
5497
5498     for (label j = 0; j < nd.nNodes; ++j)
5499     {
5500         scalar Lj = 1.0;
5501
5502         for (label m = 0; m < nd.nNodes; ++m)
5503         {
5504             if (m != j)
5505             {
5506                 Lj *= (s - nd.s[m])/(nd.s[j] - nd.s[m]);
5507             }
5508         }
5509
5510         result += Lj*nd.v[j];
5511     }
5512
5513     return result;
5514 }
```

Listing 22: El interpolante, con el caso de dos nodos escrito aparte.

```
5516 void reactionRatesLinearVm
5517 (
5518     const VmDriverNodes& vmNodes,
5519     const scalar tau,
5520     const scalar dt,
5521     const scalar u1,
5522     const scalar u2,
5523     const scalar u3,
```

```

5524     scalar& du1dt,
5525     scalar& du2dt,
5526     scalar& du3dt
5527 )
5528 {
5529     const scalar alpha =
5530         dt > SMALL
5531         ? min(max(tau/dt, scalar(0.0)), scalar(1.0))
5532         : scalar(1.0);
5533
5534     const scalar VmTau = interpolateVmDriver(vmNodes, alpha);
5535
5536     reactionRates(VmTau, u1, u2, u3, du1dt, du2dt, du3dt);
5537 }

```

Listing 23: Dónde el driver entra a las EDO.

12. Reconstrucción a los puntos de integración

Con alto orden, V_m y los estados viven en los centros de celda pero la corriente iónica se evalúa en los puntos de Gauss. Estas rutinas hacen la reconstrucción MLS de ida y el promedio de vuelta.

`reconstructStatesAtIionIntegrationPoints` lleva un limitador de Barth–Jespersen, acotando la reconstrucción al rango de los vecinos. No es cosmético: sin él, el sobrepaso de alto orden sobre estados rígidos desborda.

```

5131 void reconstructVmAtIionIntegrationPoints
5132 (
5133     const volScalarField& Vm,
5134     const Switch useHighOrderVm,
5135     const highOrderInterp& LREInterp_Vm,
5136     const highOrderInterp& LREInterp_Iion,
5137     scalarField& VmIntegrationPoints,
5138     const Switch useOpenMP,
5139     const label openMPThreshold,
5140     const Switch profileTimings,
5141     FineGrainedTimings& fineTimings
5142 )
5143 {
5144     const fvMesh& mesh = Vm.mesh();
5145     const vectorField& C = mesh.C();
5146     const bool twoD = mesh.nGeometricD() == 2;
5147     const label nCells = mesh.nCells();
5148
5149     const CompactListList<point>& cellIionQuadP =
5150         LREInterp_Iion.cellQuadPoints();
5151     // Where each cell's quadrature points start in the flat array. Taken
5152     // from the container rather than counted along the loop: the running
5153     // counter this replaces was a dependency between iterations, and was
5154     // the only thing making either branch unparallelisable.
5155     const labelList& ipOffsets = cellIionQuadP.offsets();
5156
5157     const scalarField& VmI = Vm.primitiveField();
5158
5159     if (useHighOrderVm)
5160     {
5161         const label order = LREInterp_Vm.order();
5162
5163         const auto tDerivStart = std::chrono::steady_clock::now();
5164
5165         // Halo exchange: MPI, once, outside the threaded region.
5166         const auto& stencilData = LREInterp_Vm.mls().stencilData();
5167         const globalIndex& globalCells = stencilData.globalCells();
5168         const Map<FixedList<label, 2>>& remoteLoc =
5169             stencilData.remoteCellLocation();
5170         const CompactListList<label>& stencils = stencilData.cellsStencil();
5171
5172         const List<Field<scalar>> VmRemote =
5173             stencilData.remoteFieldPerProc(VmI);
5174

```

```

5175 // Coefficients of the local fit, already built and cached by the
5176 // library. The Hessian and third-derivative tables are empty below
5177 // the order that defines them, hence the guards.
5178 const CompactListList<vector>& gradCoeffs =
5179     LREInterp_Vm.QRGradCoeffs();
5180 const CompactListList<symmTensor>& hessCoeffs =
5181     LREInterp_Vm.cellHessianCoeffs();
5182 const CompactListList<highOrderInterp::symmTensor3Order>&
5183     thirdCoeffs = LREInterp_Vm.cellThirdDerivCoeffs();
5184
5185 const bool haveHess = (order >= 2);
5186 const bool haveThird = (order >= 3);
5187
5188 const auto tExpandStart = std::chrono::steady_clock::now();
5189
5190 // One pass: gather the stencil once, form every derivative, expand

```

Listing 24: Reconstrucción de Vm. Recortada. (recortado; sigue en el fuente.)

```

5303 void averageIntegrationPointFieldToCells
5304 (
5305     const scalarField& integrationPointValues,
5306     const highOrderInterp& LREInterp_Iion,
5307     volScalarField& field
5308 )
5309 {
5310     const fvMesh& mesh = field.mesh();
5311     const CompactListList<scalar>& cellIionQuadW =
5312         LREInterp_Iion.cellQuadWeightPhysical();
5313
5314     scalarField& cellValues = field.primitiveFieldRef();
5315     label integrationPointI = 0;
5316
5317     forAll(mesh.cells(), cellI)
5318     {
5319         scalar valueBar = 0.0;
5320         scalar wSum = 0.0;
5321
5322         forAll(cellIionQuadW[cellI], qI)
5323         {
5324             const scalar w = cellIionQuadW[cellI][qI];
5325             valueBar += w*integrationPointValues[integrationPointI];
5326             wSum += w;
5327             ++integrationPointI;
5328         }
5329
5330         cellValues[cellI] = valueBar/max(wSum, SMALL);
5331     }
5332
5333     field.correctBoundaryConditions();
5334 }

```

Listing 25: El promedio de vuelta a celdas.

13. Integración de las EDO, y el paralelismo por hilos

`advanceStateODE` avanza los estados en cada celda o punto de integración, con RK4 o RK45 adaptativo según el diccionario.

Acá están los **únicos** lazos con OpenMP de todo el archivo. Es importante para entender el rendimiento: los hilos reparten *estas* iteraciones y nada más — **el ensamblado no está paralelizado con hilos**, sólo por MPI. La región funciona porque cada punto integra su propia EDO sin hablar con ningún otro, y por eso el resultado es **bit-idéntico** con cualquier cantidad de hilos, cosa que **E20** mide en el otro solver dando exactamente cero.

El umbral por debajo del cual la región no se abre llega como parámetro desde la clave `stateODEOpenMPThreshold`, en las **22** llamadas que lo pasan. Esto figuró un tiempo como ítem abierto — se creía que el valor estaba fijo en 256 en seis lazos y que la clave no hacía nada — y al ir a arreglarlo resultó que ya

estaba conectado y que lo desactualizado era el registro. Lo que sí se hizo fue **quitar los argumentos por defecto** de las seis firmas que reciben el umbral, para que una llamada nueva que se olvide de pasarlo no compile, en vez de correr en silencio con un 256 escrito en la firma.

```

5697 void advanceStateODE
5698 (
5699     const VmDriverNodes& vmNodes,
5700     const scalar u1Old,
5701     const scalar u2Old,
5702     const scalar u3Old,
5703     const scalar dt,
5704     const word& stateODESolver,
5705     const scalar stateODEInitialStep,
5706     const scalar stateODEAbsTol,
5707     const scalar stateODERelTol,
5708     const label stateODEMaxSteps,
5709     scalar& u1New,
5710     scalar& u2New,
5711     scalar& u3New
5712 )
5713 {
5714     if (dt <= SMALL)
5715     {
5716         u1New = u1Old;
5717         u2New = u2Old;
5718         u3New = u3Old;
5719         return;
5720     }
5721
5722     if (stateODESolver == "Euler")
5723     {
5724         scalar du1 = 0.0, du2 = 0.0, du3 = 0.0;
5725         reactionRatesLinearVm
5726         (
5727             vmNodes, 0.0, dt,
5728             u1Old, u2Old, u3Old,
5729             du1, du2, du3
5730         );
5731
5732         u1New = u1Old + dt*du1;
5733         u2New = u2Old + dt*du2;
5734         u3New = u3Old + dt*du3;
5735         return;
5736     }
5737
5738     if (stateODESolver == "RK4")
5739     {
5740         rk4StateStep
5741         (
5742             vmNodes, 0.0, dt, dt,
5743             u1Old, u2Old, u3Old,
5744             u1New, u2New, u3New
5745         );
5746         return;
5747     }
5748
5749     // Validated at start-up in createFields.H, so this is a guard, not a
5750     // reachable branch.
5751     if (stateODESolver != "RK45")
5752     {
5753         FatalErrorInFunction
5754         << "Unknown state ODE solver: " << stateODESolver << nl
5755         << "Valid options are RK45, RK4 and Euler"
5756         << exit(FatalError);

```

Listing 26: El avance de los estados. Recortado. *(recortado; sigue en el fuente.)*

```

5600 void rkf45StateStep
5601 (
5602     const VmDriverNodes& vmNodes,
5603     const scalar tau,
5604     const scalar h,
5605     const scalar dt,
5606     const scalar u1,
5607     const scalar u2,

```

```

5608     const scalar u3,
5609     scalar& u1Fifth,
5610     scalar& u2Fifth,
5611     scalar& u3Fifth,
5612     const scalar stateODEAbsTol,
5613     const scalar stateODERelTol,
5614     scalar& err
5615 )
5616 {
5617     scalar k11 = 0.0, k12 = 0.0, k13 = 0.0;
5618     scalar k21 = 0.0, k22 = 0.0, k23 = 0.0;
5619     scalar k31 = 0.0, k32 = 0.0, k33 = 0.0;
5620     scalar k41 = 0.0, k42 = 0.0, k43 = 0.0;
5621     scalar k51 = 0.0, k52 = 0.0, k53 = 0.0;
5622     scalar k61 = 0.0, k62 = 0.0, k63 = 0.0;
5623
5624     reactionRatesLinearVm
5625     (
5626         vmNodes, tau, dt,
5627         u1, u2, u3,
5628         k11, k12, k13
5629     );
5630     reactionRatesLinearVm
5631     (
5632         vmNodes, tau + h/5.0, dt,
5633         u1 + h*(1.0/5.0)*k11,
5634         u2 + h*(1.0/5.0)*k12,
5635         u3 + h*(1.0/5.0)*k13,
5636         k21, k22, k23
5637     );
5638     reactionRatesLinearVm
5639     (
5640         vmNodes, tau + 3.0*h/10.0, dt,
5641         u1 + h*((3.0/40.0)*k11 + (9.0/40.0)*k21),
5642         u2 + h*((3.0/40.0)*k12 + (9.0/40.0)*k22),
5643         u3 + h*((3.0/40.0)*k13 + (9.0/40.0)*k23),
5644         k31, k32, k33
5645     );
5646     reactionRatesLinearVm
5647     (
5648         vmNodes, tau + 3.0*h/5.0, dt,
5649         u1 + h*((3.0/10.0)*k11 - (9.0/10.0)*k21 + (6.0/5.0)*k31),
5650         u2 + h*((3.0/10.0)*k12 - (9.0/10.0)*k22 + (6.0/5.0)*k32),
5651         u3 + h*((3.0/10.0)*k13 - (9.0/10.0)*k23 + (6.0/5.0)*k33),
5652         k41, k42, k43
5653     );
5654     reactionRatesLinearVm
5655     (
5656         vmNodes, tau + h, dt,
5657         u1 + h*((-11.0/54.0)*k11 + (5.0/2.0)*k21 - (70.0/27.0)*k31 + (35.0/27.0)*k41),
5658         u2 + h*((-11.0/54.0)*k12 + (5.0/2.0)*k22 - (70.0/27.0)*k32 + (35.0/27.0)*k42),
5659         u3 + h*((-11.0/54.0)*k13 + (5.0/2.0)*k23 - (70.0/27.0)*k33 + (35.0/27.0)*k43),

```

Listing 27: El paso adaptativo. Recortado. (*recortado; sigue en el fuente.*)

14. main(): montaje

`main()` arranca leyendo la malla y los diccionarios, construye los operadores de alto orden, y ensambla M y K **una sola vez** — son constantes, porque la no linealidad está en el término iónico y no en la difusión. Por eso en el desglose de tiempos el `setup` aparece separado del `loop`.

Es también donde se fija `gRowStart`, y donde viven los dos volcados de diagnóstico que la verificación usa, los dos detrás de una variable de entorno y gratis cuando están apagados: `CF_DUMP_K` escribe K como tripletes (fila global, columna global, valor) y `CF_DUMP_STENCILS` escribe la membresía de los stencils. Son la base de M09, M10, M11 y M15.

Lo primero de `main()`, antes de cualquier asignación grande, son tres llamadas a `mallopt`, y no son cosmética: la construcción de los operadores corre unos **2.3 millones de factorizaciones QR** sobre

una malla de tetraedros 3D con $N=40$ y $p3$, y sin ajustar el asignador glibc crece el heap en varios GB de fragmentos que no devuelve y la corrida muere por falta de memoria en una máquina de 16 GB. `M_ARENA_MAX=2` evita que glibc cree una arena por hilo de OpenMP — el montaje es serial, así que las arenas sólo inflan el RSS —, y los dos umbrales en 64 KB hacen que las asignaciones medianas pasen por `mmap` y que el relleno del heap vuelva al sistema operativo. Es el mismo techo de memoria que **M17** mide y el que explica por qué en 3D no se pasa de $N=30$.

```

6650 int main(int argc, char* argv[])
6651 {
6652 #ifdef __GLIBC__
6653     // Tighten the glibc allocator before any large allocation. The LRE
6654     // construction runs ~2.3M QR factorisations on a 3D unstructured tet
6655     // mesh with N=40 / p3; without these tunables glibc grows the heap by
6656     // several GB of unreclaimed fragments and the run OOMs on a 16 GB box.
6657     // - M_ARENA_MAX=2: prevents glibc from creating ~64 arenas (one set
6658     //   per ODE OpenMP thread). LRE setup is serial; multiple arenas
6659     //   just inflate the RSS.
6660     // - M_MMAP_THRESHOLD=64 KB: routes mid-sized allocations (Eigen
6661     //   temporaries during QR) through mmap so they are returned to
6662     //   the kernel individually when freed.
6663     // - M_TRIM_THRESHOLD=64 KB: lets glibc release sbrk-heap padding
6664     //   back to the OS more aggressively.
6665     mallopt(M_ARENA_MAX, 2);
6666     mallopt(M_MMAP_THRESHOLD, 64*1024);
6667     mallopt(M_TRIM_THRESHOLD, 64*1024);
6668 #endif
6669
6670     #include "setRootCaseLists.H"
6671     PetscSession petscSession(argc, argv);
6672     #include "createTime.H"
6673     #include "createMesh.H"

```

Listing 28: El arranque.

```

6653     // Tighten the glibc allocator before any large allocation. The LRE
6654     // construction runs ~2.3M QR factorisations on a 3D unstructured tet
6655     // mesh with N=40 / p3; without these tunables glibc grows the heap by
6656     // several GB of unreclaimed fragments and the run OOMs on a 16 GB box.
6657     // - M_ARENA_MAX=2: prevents glibc from creating ~64 arenas (one set
6658     //   per ODE OpenMP thread). LRE setup is serial; multiple arenas
6659     //   just inflate the RSS.
6660     // - M_MMAP_THRESHOLD=64 KB: routes mid-sized allocations (Eigen
6661     //   temporaries during QR) through mmap so they are returned to
6662     //   the kernel individually when freed.
6663     // - M_TRIM_THRESHOLD=64 KB: lets glibc release sbrk-heap padding
6664     //   back to the OS more aggressively.
6665     mallopt(M_ARENA_MAX, 2);
6666     mallopt(M_MMAP_THRESHOLD, 64*1024);
6667     mallopt(M_TRIM_THRESHOLD, 64*1024);

```

Listing 29: El asignador, ajustado antes de la primera asignación grande.

```

6959 gRowStart = LREInterp_Vm.globalCells().localStart();
6960 gRowStartSet = true;
6961
6962 Info<< "Distributed linear algebra: rows ["
6963     << gRowStart << ", " << gRowStart + mesh.nCells() << ") of "
6964     << LREInterp_Vm.globalCells().totalSize() << " global cells" << endl;
6965
6966 SpMat M;

```

Listing 30: Dónde se fija el offset global de fila.

```

2085 void logMemoryCheckpoint(const char* stage)
2086 {
2087     const scalar peakMemoryKB = peakResidentSetSizeKB();
2088     Info<< "Memory checkpoint [" << stage << "]: peakRSS_MB = "
2089         << peakMemoryKB/1024.0 << endl;
2090 }

```

Listing 31: Los checkpoints de memoria que M17 usa para ajustar el techo.

15. `main()`: el lazo temporal y los métodos no lineales

Cada paso resuelve un sistema no lineal, y hay tres métodos: **Picard** con relajación, **JFNK** — Newton sin jacobiano explícito, con el producto matriz-vector aproximado por diferencias — y **diagonalIion**, que linealiza la corriente iónica por su derivada diagonal. **M08** verifica que los tres lleguen al mismo lugar, y lo que exige no es una cota absoluta sino una *dirección de cambio*: que la dispersión entre los tres caiga en cada escalón de una escalera de `implicitTolerance`. Eso dice exactamente lo que se quiere afirmar — que lo que los separa es el piso del solve lineal y no una diferencia estructural — y no lleva ninguna constante. Su versión anterior derivaba una cota de `nonlinearVmTolerance`, y estaba indexada a una perilla que no gobernaba la cantidad que acotaba: apretarla cuatro órdenes no movía la discrepancia ni en el décimo dígito.

Ese test tenía además un punto ciego, y detrás había un defecto real en JFNK. Su lazo evalúa la convergencia dos veces por iteración, al tope y al final, y la del tope necesita los incrementos que acumuló la iteración *anterior*. Los recalculaba en el lugar contra una copia tomada unas líneas antes, con una sola re-evaluación del *mismo* punto en medio, así que valían cero por construcción: cuatro de las cinco tolerancias comparaban cero contra un número positivo y el test se reducía al residual acoplado. Consecuencia medida en 3-D: en 24 de 50 pasos JFNK devolvía la extrapolación inicial sin dar un paso de Newton y con cero llamadas al KSP. Se arregló guardando los incrementos medidos al final de cada iteración; `max_coupled_reLL2` cayó de 9.493e-09 a 5.655e-16 y Picard y `diagonalIion` quedaron bit a bit idénticos.

Al agotar las iteraciones sin converger hay dos salidas posibles, y **los tres métodos las eligen ahora con la misma clave**: `false` descarta el paso y vuelve a los valores del inicio, `true` se queda con el iterado y lo dice. Antes Picard tenía su propia rama que aceptaba el último iterado *siempre*, así que el mismo solver tenía dos semánticas de error según el método. Se eliminó esa rama en vez de agregarle un rollback propio: las otras dos ya implementan las dos salidas, así que Picard cae solo en la correcta.

Medido forzando la no convergencia con `nonlinearIterations` 1 y tolerancias en 1e-30, que es la única forma de hacerlo disparar — en los barridos de convergencia no ocurre nunca: con `false` el error final es **4.998e-04**, que es el de no haber avanzado el paso, y con `true` es **5.023e-06**. El rollback efectivamente revierte.

Un nombre de método inválido es `FatalError` y no un silencioso repliegue sobre Picard, que es lo que hacía el solver electro hasta que **E18** lo señaló.

La condición de parada del lazo tenía un error, y era invisible. El tiempo acumulado es una suma en punto flotante de Δt , y puede caer por debajo de `endTime` *por más que* `SMALL`: 160 pasos de $3,2 \times 10^{-3}$ suman 0,51199999999999868, a $1,3 \times 10^{-15}$ de 0,512. Con la condición vieja, `endTime` - `SMALL`, el lazo daba un paso de más y la corrida terminaba en 0,5152 sin avisar nada. Eso no es cosmético para un estudio temporal: los campos que se comparan entre corridas con distinto Δt dejan de ser del mismo instante. Ahora la condición es `endTime` - $\Delta t/2$, exacta porque acá Δt es constante. Lo encontró la guarda de un test de la suite, no una revisión del código, y el mismo error sigue presente en el solver electro.

```
6892  const bool usePicard = nonlinearMethod == "Picard";
6893  const bool useDiagonalIion = nonlinearMethod == "diagonalIion";
6894  const bool useJFNK = nonlinearMethod == "JFNK";
6895
6896  if (!usePicard && !useDiagonalIion && !useJFNK)
6897  {
6898      FatalErrorInFunction
6899          << "Unknown nonlinearMethod '" << nonlinearMethod << "'. "
6900          << "Valid options are Picard, JFNK and diagonalIion (exact"
6901          << " spelling, case-sensitive)."
6902          << exit(FatalError);
6903  }
```

Listing 32: El despacho de método no lineal, y el rechazo de un nombre inválido.

```

9678 // Picard used to have its own branch here
9679 // that accepted the last iterate unconditionally, so the same solver
9680 // had two error semantics depending on the method. All three now obey
9681 // nonlinearAcceptUnconverged: false rolls the step back, true keeps the
9682 // iterate and says so.
9683 if (!nonlinearConverged && !nonlinearAcceptUnconverged)
9684 {
9685     WarningInFunction
9686     << "Nonlinear solver did not converge at t = " << t
9687     << " after " << nonlinearIters << " iterations."
9688     << " Rolling back Vm, Iion and states to the values"
9689     << " at the beginning of the time step." << nl
9690     << "Set 'nonlinearAcceptUnconverged true;' in"
9691     << " spatialIntegrationProperties to accept unconverged"
9692     << " iterates instead." << endl;
9693
9694     VmGuess.primitiveFieldRef() = VmOld.primitiveField();
9695     applyExactVmBoundaryValues(VmGuess, t, dim);
9696     u1Guess.primitiveFieldRef() = u1Old.primitiveField();
9697     u2Guess.primitiveFieldRef() = u2Old.primitiveField();
9698     u3Guess.primitiveFieldRef() = u3Old.primitiveField();
9699     updateStateBoundaryValues(u1Guess, u2Guess, u3Guess, t, dim);
9700     u1Guess.correctBoundaryConditions();
9701     u2Guess.correctBoundaryConditions();
9702     u3Guess.correctBoundaryConditions();
9703     IionGuess.primitiveFieldRef() = Iion.primitiveField();
9704     IionGuess.correctBoundaryConditions();
9705     nonlinearRolledBack = true;

```

Listing 33: Las dos salidas al agotar las iteraciones, ahora comunes a los tres métodos.

```

7523 // Stop within half a step of endTime, not within
7524 // SMALL. The accumulated time is a floating-point sum of dt, and it can land
7525 // just BELOW endTime by more than SMALL (1e-15): 160 steps of 3.2e-3 sum to
7526 // 0.51199999999999868, which is 1.3e-15 short of 0.512. The old test then
7527 // took one extra step and the run finished at 0.5152 with no warning - so a
7528 // field or an error "at endTime" belonged to a different instant, and a
7529 // temporal-order study comparing runs at different dt compared fields at
7530 // different times. Half a step is exact because dt is constant here, and it
7531 // leaves every run that was already correct with the same step count:
7532 // checked across 159 151 existing summaries, none of which had an extra step.
7533 while (runTime.value() < runTime.endTime().value() - 0.5*dt)
7534 {

```

Listing 34: El lazo que ya no da un paso de más, y por qué.

```

8951 // The increments measured at the END of the previous corr, which are
8952 // what the check at the top of the next one is entitled to look at.
8953 //
8954 // That check used to recompute them against a copy taken two lines
8955 // further up, with only residualFor(x, true) in between - and that
8956 // call re-evaluates the same x, so the "increments" were the
8957 // difference between two evaluations of one point, i.e. identically
8958 // zero. Four of the five tolerance terms therefore compared zero
8959 // against a positive number and could never fail: the test reduced
8960 // to 'newtonResidual <= nonlinearTolerance' alone, and JFNK returned
8961 // the jfnkInitGuessOrder extrapolation without taking a single
8962 // Newton step whenever that extrapolation happened to land under the
8963 // coupled tolerance. Measured on M08 in 3-D: 24 of 50 timesteps
8964 // exited that way, 6.3e-08 from the solution the other two
8965 // nonlinear methods agree on, and no tolerance the user could set
8966 // made any difference - only nonlinearTolerance did.
8967 //
8968 // GREAT on entry means the first pass can never exit there, so at
8969 // least one Newton step is always taken. Verified by construction
8970 // against minNonlinearIterations = 2, which forces the same thing
8971 // from the outside and closes the gap to 2.0e-11.
8972 scalar VmIncrPrevIter = GREAT;
8973 scalar u1IncrPrevIter = GREAT;
8974 scalar u2IncrPrevIter = GREAT;
8975 scalar u3IncrPrevIter = GREAT;
8976 scalar IionIncrPrevIter = GREAT;

```

Listing 35: Por qué los incrementos del chequeo del tope de JFNK viven fuera del lazo: recalculados adentro valían cero y mataban cuatro tolerancias.

16. Índice de funciones

Todas las funciones del espacio de nombres anónimo, en orden de aparición. Las secciones anteriores recorren las principales; ésta es el mapa completo para ubicarse en el archivo.

línea	función	líneas
110	globalRowStart	15
133	checkPetscError	10
155	buildDistributedMat	85
273	clear	15
289	reset	19
359	requireOneOf	31
401	usesPetscBackend	5
425	resolvePetscKspPc	18
454	petscKspTypeForLinearSolver	12
476	petscParallelPcTypeName	13
492	isGmresType	4
548	copyEigVecToPetscVec	12
562	copyPetscVecToEigVec	16
582	setPetscOptionsPrefix	11
818	updateValues	65
884	solve	54
951	petscShellMatMult	17
978	petscShellPCApply	17
1066	initialise	153
1419	computeF	25
1434	if	4
1447	computeG	21
1472	exactVm	4
1479	exactU1	4
1485	exactU2	4
1492	exactU3	4
1499	exactFieldValue	20
1523	reactionRates	18
1546	ionicCurrentPDE	15
1566	vmSourcePDE	13
1584	computeBeta	24
1611	estimatedN	8
1624	characteristicDx	35
1664	volumeWeightedL1	7
1674	volumeWeightedL2	7
1683	linfNorm	5
1692	cellErrorNorm	4
1781	esdirk3Tableau	37
1821	timeSchemeCoeffs	71
1895	normalizedMassMatrixType	19
1915	computeFieldErrorSummary	9
1927	quadraticForm	14
1949	reconstructFromTaylor	24
1977	fieldToEigVec	12
1992	sourceToEigVec	4
2000	eigVecToField	9

línea	función	líneas
2012	relativeL2Difference	29
2049	gSquaredNorm	6
2058	gNorm	4
2065	relativeL2Norm	4
2071	peakResidentSetSizeKB	10
2085	logMemoryCheckpoint	6
2093	addDiagonalToMatrix	15
2110	solveGMRES	95
2207	solveLeftPreconditionedGMRES	97
2308	addTripletIfNeeded	13
2326	assembleDiagonalMassMatrix	20
2354	assembleConsistentMassMatrixHO	222
2581	orthogonalDiffusionCoeff	14
2598	normalDiffusivity	4
2640	nonOrthFluxVector	37
2682	stabilisationFaceCoeff	17
2703	addCellGradientDotCoeffs	46
2759	cellTaylorEntryCoeff	31
2799	buildCellTaylorExtrapolationRow	32
2834	addRemoteRowCoeffs	19
2860	buildCellGradientDotRow	27
2904	exchangeCoupledFaceRows	78
2988	addCellTaylorExtrapolationCoeffs	46
3045	addRhieChowInternalJumpCoeffs	40
3093	addRhieChowCoupledJumpCoeffs	40
3137	addRhieChowBoundaryJumpCoeffs	22
3175	assembleStandardOrthogonalStiffnessMatrix	405
3594	assembleHighOrderStiffnessMatrix	506
4105	assembleStandardOrthogonalBoundaryVector	69
4178	assembleHighOrderBoundaryVector	90
4272	solveSparseSystemEigen	72
4351	solveSparseSystem	74
4429	appendTetQuadraturePointsAndWeights	58
4492	cellQuadraturePointsAndWeights	71
4567	updateStateBoundaryValues	36
4607	applyExactVmBoundaryValues	32
4642	fillExactFields	60
4708	computeHighOrderLaplacian	85
4796	computeCellCentredIion	34
4832	computeCellCentredVmSource	34
4871	stencilValue	17
4913	reconstructStatesAtIionIntegrationPoints	189
5131	reconstructVmAtIionIntegrationPoints	168
5303	averageIntegrationPointFieldToCells	32
5336	clampPhysical	13
5353	initialiseIionIntegrationPointStates	26
5383	computeIionFromIntegrationPoints	32
5417	computeVmSourceFromIntegrationPoints	32
5482	interpolateVmDriver	33
5516	reactionRatesLinearVm	22
5541	rk4StateStep	54

línea	función	líneas
5600	rkf45StateStep	92
5697	advanceStateODE	125
5825	updateIntegrationPointStatesODE	56
5909	updateStateFieldsODE	72
5982	computeGaussReconstructedError	109
6092	cellAsReconstructionError	14
6107	computeIntegrationPointStateError	65
6650	main	3495